Kumpas.AdminWeb Beginner File-by-File Explanation

Kumpas.AdminWeb Beginner Explanation

A longer, simpler, file-by-file guide to the KUMPAS admin website. This explains what each file does, why it exists, and how the pieces connect when an admin logs in, views dashboards, manages accounts, reads conversations, and prints reports.

Project folder: Kumpas.AdminWeb

Generated: April 18, 2026

Note: Real passwords, database strings, Supabase keys, and service-role tokens are intentionally not copied into this guide.

Start Here: What This App Is

Kumpas.AdminWeb is an admin website built with ASP.NET Core MVC. In simple terms, it is a web dashboard for administrators of the KUMPAS system.

The app lets an administrator log in, open a dashboard, manage users, activate or deactivate accounts, update passwords through Supabase Auth, view conversations, delete conversation sessions, generate reports, and monitor translation-service records.

Beginner MVC Explanation

A browser asks for a page. A controller receives the request. The controller asks a service or the database context for data. The data is placed into a view model. A Razor view uses that view model to create HTML. The browser displays the HTML.

Word | Simple meaning in this project

Model | A C# class that represents a database table row, such as Profile or ChatMessage.

ViewModel | A C# class made only for a page, such as DashboardViewModel.

View | A .cshtml Razor file that creates HTML for the browser.

Controller | A C# class that handles a URL or form submission.

Service | A C# class that contains reusable work, such as querying accounts or calling Supabase.

DbContext | The EF Core object that connects C# to the PostgreSQL database.

Overall Request Flow

Browser -> URL or form submit -> Controller action -> Service/database -> ViewModel -> Razor

View -> HTML page

For example, when the admin opens the accounts page, the request goes to AccountsController.Index. That action calls AccountService.GetAccountsAsync. The service queries profiles and auth users, returns a ManageAccountsViewModel, and Views/Accounts/Index.cshtml displays the account table.

Root Files

Program.cs

What it is: The startup file. This is where the web application is assembled before it starts listening for requests.

```
builder.Services.AddDbContext<KumpasDbContext>(options =>
options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));
```

This registers the database connection. UseNpgsql means the app uses PostgreSQL.

```
builder.Services.AddAuthentication(...).AddCookie(options =>
{
options.LoginPath = "/Auth/Login";
options.Cookie.Name = "Kumpas.Admin.Auth";
options.ExpireTimeSpan = TimeSpan.FromHours(8);
});
```

This sets up cookie authentication. After a successful login, the browser receives a cookie. On later requests, ASP.NET reads that cookie and knows who the user is.

```
options.AddPolicy("AdminOnly", policy =>
{
policy.RequireAuthenticatedUser();
policy.RequireClaim("UserType", "Admin");
});
```

This creates the rule used by admin pages. A person must be signed in and must have the claim UserType = Admin.

Kumpas.AdminWeb.csproj

What it is: The project file. It tells .NET what type of app this is and what packages it needs.

- Microsoft.NET.Sdk.Web means this is an ASP.NET web app.

- TargetFramework net9.0 means it targets .NET 9.

- Microsoft.EntityFrameworkCore lets the app query the database through C# objects.

- Npgsql.EntityFrameworkCore.PostgreSQL lets EF Core talk to PostgreSQL.

appsettings.json

What it is: The main configuration file.

- ConnectionStrings:DefaultConnection: database connection information.

- Supabase: URL, anon key, and service-role key used by SupabaseAuthService.

- DevelopmentSeed: whether sample development logs should be inserted.

- ModelAssets: AR/model provider, URL, and status shown in reports.

- Logging: how much logging ASP.NET and EF Core should produce.

Security reminder: database passwords and Supabase service-role keys should not be stored in committed source code. Use user secrets, environment variables, or a secure secret manager.

appsettings.Development.json

What it is: Extra settings used when running the app in Development mode. It sets project logging to Debug so developers can see more detail while testing locally.

Properties/launchSettings.json

What it is: Local launch settings used by Visual Studio or dotnet run. It normally defines local URLs, HTTPS ports, and development environment variables. It is mainly for local development, not the main production server configuration.

Data Folder

Data/KumpasDbContext.cs

What it is: The database map for the app. EF Core uses this class to understand tables, columns, keys, and relationships.

```
public DbSet<AuthUser> AuthUsers => Set<AuthUser>();
public DbSet<Profile> Profiles => Set<Profile>();
public DbSet<ChatSession> ChatSessions => Set<ChatSession>();
```

```
public DbSet<ChatMessage> ChatMessages => Set<ChatMessage>();
public DbSet<GestureLibrary> GestureLibraries => Set<GestureLibrary>();
public DbSet<ArModel> ArModels => Set<ArModel>();
public DbSet<GestureRecognitionData> GestureRecognitionData =>
Set<GestureRecognitionData>();
public DbSet<SystemLog> SystemLogs => Set<SystemLog>();
```

Each DbSet acts like a table. For example, dbContext.Profiles means the app is working with rows from the profiles table.

OnModelCreating: This method maps C# names to database names. Example: Profile.FirstName maps to first_name. It also defines relationships such as a profile belonging to an auth user, chat sessions having two users, and messages belonging to a session.

Data/SeedData.cs

What it is: A background startup service that can add sample log rows during development.

- It runs when the app starts because Program.cs registers it with AddHostedService<SeedData>().

- It only continues if the app is in Development mode and DevelopmentSeed:Enabled is true.

- It checks whether SystemLogs has any rows.

- If no rows exist, it inserts two sample logs.

- If the database schema does not accept the insert, it logs a warning instead of crashing.

Database Folder

Database/seed-test-data.sql

What it is: A small SQL script intended to insert a test row into public.system_logs.

```
insert into public.system_logs (id, created_at)
select gen_random_uuid(), now()
where not exists (select 1 from public.system_logs);
```

This means: insert one row with a generated ID and the current time, but only if the table has no rows yet. ASP.NET does not automatically run this unless a developer or database tool runs it.

Models Folder

Models/AuthUser.cs
Represents: A user account from Supabase Auth, mapped to auth.users. It stores Id, Email, EncryptedPassword, and a linked Profile.

Models/Profile.cs
Represents: The app profile for a user. It stores name, UserType, IsActive, and created date. This file matters because admin access depends on UserType and IsActive.

Models/ChatSession.cs
Represents: One conversation session between two users. It has User1Id, User2Id, RoomCode, deleted flags, and a collection of Messages.

Models/ChatMessage.cs
Represents: One message inside a conversation. SessionId connects it to a session, SenderId connects it to a profile, and optional GestureId means it is a gesture message.

Models/GestureLibrary.cs
Represents: A known sign or gesture. It stores the gesture name, type, description, and category, plus relationships to messages, AR models, and recognition data.

Models/ArModel.cs
Represents: AR model and animation file paths for a gesture. It links back to GestureLibrary through GestureId.

Models/GestureRecognitionData.cs
Represents: Data used for recognizing gestures. It stores image path, video path, and JSON keypoint data.

Models/SystemLog.cs
Represents: One system log entry, with level, module/source, message, optional stack trace, timestamp, and optional user profile.

Models/ErrorViewModel.cs
Represents: Simple data for the error page. ShowRequestId is true only when a request ID exists.

Services Folder

Services/ISupabaseAuthService.cs

What it is: An interface. It describes what the Supabase auth service must be able to do.

- SignInAsync: checks email and password with Supabase.

- UpdatePasswordAsync: changes a user’s password using Supabase admin API.

- DeleteUserAsync: deletes a user from Supabase Auth.

SupabaseLoginResult and SupabaseOperationResult are small result objects that say whether the operation succeeded and optionally include an error message.

Services/SupabaseOptions.cs

What it is: A settings class for Supabase configuration. ASP.NET fills Url, AnonKey, and ServiceRoleKey from the Supabase section of appsettings.json.

Services/SupabaseAuthService.cs

What it is: The class that talks to Supabase Auth using HTTP requests.

SignInAsync: Sends email and password to Supabase at /auth/v1/token?grant_type=password. If login succeeds, it reads the returned user ID and access token from JSON.

UpdatePasswordAsync: Uses the service-role key to update a user's password through Supabase's admin endpoint.

DeleteUserAsync: Uses the service-role key to delete a Supabase auth user.

Beginner note: The service-role key is powerful. It can perform admin actions, so it must be protected carefully.

Services/AccountService.cs

What it is: The service for account list, account details, activation, profile updates, and profile deletion.

GetAccountsAsync: Joins profiles and auth.users. Then it applies search, status, user-type filters, and pagination. It always uses 10 rows per page.

GetAccountDetailsAsync: Loads one account and calculates conversation count, message count, last conversation date, and five recent conversations.

SetActiveStatusAsync: Finds a profile and changes IsActive.

UpdateProfileAsync: Updates first and last name.

DeleteProfileAsync: Removes the profile row from the database.

Services/ConversationService.cs

What it is: The service for listing, viewing, and deleting conversations.

GetSessionsAsync: Loads sessions with User1, User2, and Messages. It can filter by date range and search by room code or participant name.

GetSessionDetailsAsync: Loads one session, including messages and sender names. It labels each message as Gesture if GestureId has a value, otherwise Text.

DeleteSessionAsync: Deletes all messages for the session, then deletes the session itself.

Services/AdminAnalyticsService.cs

What it is: A service for dashboard, reports, and translation-service monitoring data.

GetDashboardAsync: Counts total accounts, active accounts, sessions, sessions today, messages, errors today, recent accounts, recent sessions, and recent errors.

GetTranslationServicesAsync: Counts sessions and messages in a date range, gesture library items, AR models, recognition records, system logs, and average messages per session.

GetReportsAsync: Builds daily usage, top users, and message type breakdown.

Why direct SQL appears here: Some methods use NpgsqlConnection and SQL directly so the code can check whether tables or columns exist before reading system logs.

ViewModels Folder

ViewModels/LoginViewModel.cs
Used by: Login page and AuthController. It stores Email, Password, and RememberMe. Data annotations require email/password and validate email format.

ViewModels/DashboardViewModel.cs
Used by: Dashboard page. It stores totals for accounts, sessions, messages, errors, plus recent accounts, sessions, and errors.

ViewModels/AccountViewModels.cs
Used by: Account pages. It contains ManageAccountsViewModel for the list, AccountRowViewModel for one table row, AccountDetailsViewModel for the details screen, and UpdateUserPasswordViewModel for password forms.

ViewModels/ConversationViewModels.cs
Used by: Conversation pages. It contains the list model, one session row model, details model, and one message row model.

ViewModels/ReportsViewModel.cs
Used by: Reports page. It stores date filters, account/session/message totals, AR model status, error counts, generated-by data, daily usage rows, top users, message types, and pagination for top users.

ViewModels/ProfileSettingsViewModel.cs
Used by: Settings page. It stores current admin name, email, optional new password, and

confirmation password. Email is shown read-only because Supabase Auth manages it.

ViewModels/PaginationViewModel.cs
Used by: Shared pagination partial. It calculates total pages, previous/next availability, visible page numbers, route values, and text like Showing 1-10 of 45 records.

ViewModels/SystemLogItemViewModel.cs
Used by: Dashboard and translation monitoring pages. It stores log ID, level, message, source, user name, and created date.

ViewModels/TranslationServicesViewModel.cs
Used by: Translation Services page. It stores date filters, totals, average messages per session, and system log rows.

Controllers Folder

Controllers/AuthController.cs

What it does: Handles login and logout.

GET Login: Shows the login form. If the user is already logged in, it redirects to the dashboard.

POST Login: Validates the form, checks credentials with Supabase, loads the user's profile, blocks inactive accounts, blocks non-admin users, creates login claims, and signs the user in with a cookie.

UserType is saved as a claim because the AdminOnly policy checks for UserType = Admin.

Logout: Deletes the auth cookie and redirects to login.

Controllers/DashboardController.cs

What it does: Builds the dashboard page.

It counts accounts, active accounts, total sessions, sessions today, total messages, and errors today. It also loads recent accounts, recent sessions, and recent errors.

Safety check: Before reading system_logs, it checks whether the table exists. If not, the dashboard avoids crashing and simply shows zero log data.

Controllers/AccountsController.cs

What it does: Handles admin account management.

- Index: shows the searchable, filterable account list.

- Details: shows one account.

- ToggleStatus and ToggleStatusFromDetails: activate/deactivate accounts.

- UpdatePassword and UpdatePasswordFromDetails: validate password forms and call Supabase.

- Delete: deletes the Supabase auth user first, then deletes the local profile.

POST actions use [ValidateAntiForgeryToken], which helps prevent fake form submissions from other websites.

Controllers/ConversationsController.cs

What it does: Handles conversation history pages.

Index shows conversations with optional search and date filters. Details shows the messages in one conversation. Delete removes a conversation and redirects back to the list with a success or error message.

Controllers/ReportsController.cs

What it does: Builds the reports page.

It calculates account totals, session totals, message totals, AR model count, model status, error log totals, daily usage, top users, message type totals, and pagination for top users.

Beginner note: This controller has a lot of query logic. Some similar logic also exists in AdminAnalyticsService, so a future cleanup could move more reporting work into that service.

Controllers/SettingsController.cs

What it does: Lets the current admin update their own profile and optionally change password.

It reads the current user’s ID from the login claim, loads the profile, shows first name, last name, and email, and saves changes. If a new password is entered, it calls Supabase to update it.

Controllers/TranslationServicesController.cs

What it does: Shows translation-service monitoring metrics. It accepts optional date filters, calls AdminAnalyticsService.GetTranslationServicesAsync, and returns the monitoring view.

Views Folder

Views/_ViewStart.cshtml
What it does: Sets Layout = "_Layout", meaning views automatically use Views/Shared/_Layout.cshtml.

Views/_ViewImports.cshtml
What it does: Imports namespaces and MVC tag helpers. Tag helpers allow Razor attributes like asp-controller, asp-action, asp-for, and asp-validation-for.

Views/Shared/_Layout.cshtml
What it does: Creates the main page frame. Anonymous users see the login shell. Authenticated users see the sidebar, KUMPAS branding, navigation, topbar, admin name, logout button, alert messages, and page content. @RenderBody() is where each page's content is inserted.

Views/Shared/_Pagination.cshtml
What it does: Reusable pagination UI. It displays summary text, Previous button, page numbers, and Next button while preserving search/date filters.

Views/Shared/_ValidationScriptsPartial.cshtml
What it does: Loads client-side validation JavaScript. Login and Settings use it so browser-side validation can show form errors.

Views/Shared/Error.cshtml
What it does: Displays a friendly error page and can show a request ID to help developers find related logs.

Views/Auth/Login.cshtml
What it does: Shows the admin login form. It binds to LoginViewModel. Inputs like asp-for="Email" connect the HTML field to the model property.

Views/Dashboard/Index.cshtml
What it does: Shows dashboard stat cards, recent accounts, recent errors, and recent conversations using DashboardViewModel.

Views/Accounts/Index.cshtml
What it does: Shows filters for search, status, and user type. It displays an account table and includes the shared pagination partial.

Views/Accounts/Details.cshtml
What it does: Shows one user's details and provides forms to update password and activate/deactivate the account.

Views/Conversations/Index.cshtml
What it does: Shows conversation history with filters, a table, View buttons, and Delete forms. Delete forms use data-confirm so JavaScript asks before deleting.

Views/Conversations/Details.cshtml
What it does: Shows one conversation with room code, participants, time, sender names, message type, and message content.

Views/Reports/Index.cshtml
What it does: Shows report filters, a printable report section, stat cards, daily usage, top users, and message type counts. The Print Report button calls window.print().

Views/Settings/Index.cshtml
What it does: Shows the current admin's profile/settings form. The admin can update first name, last name, and optionally password. Email is read-only.

Views/TranslationServices/Index.cshtml
What it does: Shows translation service monitoring metrics and recent system logs.

wwwroot Folder

wwwroot/css/site.css

What it does: Controls the look and layout of the admin website.

It defines KUMPAS colors, backgrounds, cards, sidebar, topbar, tables, badges, buttons, forms, pagination, login screen, responsive layout, and print styles.

- :root defines reusable CSS variables like --kumpas-blue.

- .app-shell, .sidebar, and .main-shell build the main admin layout.

- .soft-card, .stat-card, and .data-table style common UI blocks.

- @media rules adjust layout for smaller screens.

- @media print hides navigation and filters when printing reports.

wwwroot/js/site.js

What it does: Adds small browser-side behavior.

The first part watches forms with data-confirm. When the form is submitted, it shows a confirmation dialog. If the admin cancels, it prevents the form from submitting.

The second part toggles an active class on collapse buttons. This helps UI controls visually show when they are active.

wwwroot/images/kumpas-logo.png
What it is: The KUMPAS logo image used in the layout and favicon area.

wwwroot/favicon.ico
What it is: A small icon browsers can show in a tab or bookmark.

wwwroot/lib
What it is: Third-party frontend libraries such as Bootstrap, jQuery, jQuery Validation, and unobtrusive validation. This guide does not explain every vendor file because those are library files, not KUMPAS-specific code.

Important Flows Explained Slowly

Flow 1: Admin Login

- The admin opens the login page.

- AuthController.Login GET returns Views/Auth/Login.cshtml.

- The admin submits email and password.

- LoginViewModel checks that email and password are present.

- SupabaseAuthService.SignInAsync sends credentials to Supabase.

- If Supabase accepts them, the app receives a user ID.

- The controller loads Profile using that user ID.

- The controller checks that the profile exists, is active, and has UserType = Admin.

- The controller creates claims and signs in using a cookie.

- The admin is redirected to the dashboard.

Flow 2: Opening the Dashboard

- The browser requests /Dashboard/Index.

- The admin cookie is checked.

- The AdminOnly policy checks for UserType = Admin.

- DashboardController.Index counts database records.

- The controller creates DashboardViewModel.

- Views/Dashboard/Index.cshtml displays stat cards and tables.

Flow 3: Managing Accounts

- The admin opens Manage Accounts.

- AccountsController.Index receives optional filters.

- AccountService.GetAccountsAsync joins profiles with auth users.

- The service filters, sorts, counts, and paginates the data.

- The view shows the account table.

- For details, the admin clicks View, which opens AccountsController.Details.

- For password changes or activation, the page submits a POST form.

Flow 4: Viewing and Deleting Conversations

- The admin opens Conversation History.

- ConversationsController.Index receives search/date filters.

- ConversationService.GetSessionsAsync loads sessions, users, and messages.

- The list view shows sessions and pagination.

- The Details page loads one session and shows all messages in time order.

- If Delete is clicked, site.js asks for confirmation.

- If confirmed, ConversationService.DeleteSessionAsync deletes messages first, then the session.

Flow 5: Generating Reports

- The admin chooses filters on the Reports page.

- ReportsController.Index converts dates into UTC start/end ranges.

- The controller counts accounts, sessions, messages, AR models, and errors.

- It groups sessions/messages by day and groups messages by sender and type.

- The data is placed in ReportsViewModel.

- The report view shows both web cards and a printable report section.

Things To Watch Out For

- Secrets in config: move database passwords and Supabase service-role keys out of committed files.

- Duplicate analytics logic: dashboard/report code appears both in services and controllers. Keeping it mostly in services would make maintenance easier.

- Delete order: account deletion removes the Supabase user before local profile cleanup. If local cleanup fails, the auth user may already be deleted.

- Null assumptions: some queries assume related profiles exist. Broken database relationships could cause page errors.

- Time zones: database filtering uses UTC-style ranges, while views show local time with ToLocalTime().

Glossary

Term | Meaning

ASP.NET Core | Microsoft’s web framework for building websites and APIs in C#.

MVC | Model-View-Controller, a way to organize web code.

Razor | HTML mixed with C# using @ syntax.

EF Core | Entity Framework Core, the database library used by the app.

LINQ | C# query syntax used to filter, sort, join, and group data.

Supabase Auth | The authentication system used for login, password updates, and user deletion.

Cookie | A small browser value used here to remember that the admin is logged in.

Claim | A stored identity value, such as user ID, email, or UserType.

Anti-forgery token | A hidden form value that helps prove a form was submitted from the real site.

Pagination | Splitting a long list into pages, such as 10 accounts per page.

Final Simple Summary

The app starts in Program.cs. It connects to PostgreSQL/Supabase through KumpasDbContext. Login is checked by SupabaseAuthService. Admin-only pages are protected by a policy that

requires UserType = Admin. Controllers handle browser requests. Services do most reusable data work. ViewModels carry prepared data to Razor views. The views display the admin pages. CSS and JavaScript make the pages look good and add small interactions.

The main idea is: request -> controller -> service/database -> view model -> view -> page.